

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO4: Evaluate model-based reinforcement learning approaches for prediction, planning, and policy optimization. (BL3, BL4, BL5)

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Technical Presentation

DURATION: 2HRS

Code of Conduct:

I ALLAPAREDDY JAHNAVI (192472214) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student A. Jahnavi

1. Dynamic Programming for Sequential Decision-Making in Reinforcement Learning

1. Introduction

Dynamic Programming (DP) is a classical approach for solving sequential decision-making problems in Reinforcement Learning when the **environment model is known**.

It uses the **Bellman equations** to break a complex decision problem into smaller subproblems.

Key idea:

Find the optimal decision for the current state by considering the immediate reward and the best possible future value.

DP mainly includes:

- Bellman Principle of Optimality
- Policy Evaluation
- Policy Improvement
- Policy Iteration
- Value Iteration

2. Problem Identification

In sequential decision-making, an RL agent must determine:

- What action should be selected?
- What is the expected future reward?
- Which policy gives the maximum cumulative reward?
- How can the optimal policy be obtained efficiently?

For example, in **robot navigation**, the robot must choose movements that eventually reach the destination while avoiding obstacles and minimizing travel cost.

Dynamic Programming solves this by repeatedly estimating state values and improving decisions.

3. Markov Decision Process

DP-based RL represents the problem using an MDP:

$$[MDP = (S, A, P, R, \gamma)]$$

Where:

- **S** – Set of states
- **A** – Set of actions
- **P** – Transition probability
- **R** – Reward function
- γ – Discount factor

4. Bellman Principle of Optimality

The Bellman Principle states that:

An optimal policy has the property that whatever the initial state and first decision are, the remaining decisions must constitute an optimal policy for the resulting state.

The Bellman optimality equation is:

$$V^*(s) = \max_{a \in \mathcal{A}} \left[R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^*(s') \right]$$

This means the optimal value of a state depends on the immediate reward plus the discounted value of future states.

5. Policy Evaluation

Policy evaluation calculates the value of following a particular policy.

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \left[R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^\pi(s') \right]$$

The value function is repeatedly updated until it converges.

Purpose: Determine how good the current policy is.

6. Policy Improvement

After evaluating a policy, the agent checks whether another action can provide a higher expected return.

$$\pi_{\text{new}}(s) = \arg \max_{a \in \mathcal{A}} \left[R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^{\pi}(s') \right]$$

Thus:

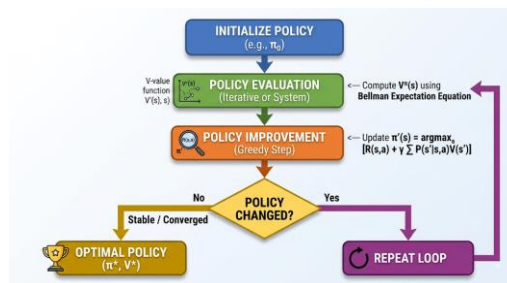
Policy Evaluation → Policy Improvement → Better Policy

7. Policy Iteration

Policy Iteration alternates between:

1. Initialize a policy.
2. Evaluate the policy.
3. Improve the policy.
4. Check whether the policy has changed.
5. If changed, repeat.
6. If unchanged, the policy is optimal.

Flow:



8. Value Iteration

Value Iteration directly updates the value function using the Bellman optimality equation.

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \left[R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V_k(s') \right]$$

After convergence, the optimal policy is extracted.

9. Value Iteration vs Policy Iteration

Feature	Value Iteration	Policy Iteration
Main operation	Value update	Policy evaluation + improvement
Policy evaluation	Implicit	Explicit
Computation per iteration	Usually lower	Usually higher
Convergence	Often more iterations	Often fewer iterations
Policy extraction	After convergence	During improvement

10. Industrial Applications

Dynamic Programming can be applied to:

- Robot path planning
- Inventory management
- Resource allocation
- Production scheduling
- Energy management
- Traffic signal optimization
- Supply-chain decision-making

Example: In smart manufacturing, DP can determine the sequence of machine operations that minimizes energy consumption while satisfying production requirements.

11. Tools and Experimental Evaluation

A simple experimental implementation can use:

- **Python**
- **NumPy**
- **Matplotlib**
- **Gymnasium**
- Grid-world environments

Performance can be evaluated using:

- Number of iterations
- Convergence speed

- Total reward
- Number of steps to reach the goal
- Computational time

12. Conclusion

Dynamic Programming provides a mathematical foundation for solving model-based RL problems. **Policy Iteration** focuses on repeatedly evaluating and improving policies, while **Value Iteration** directly updates value estimates toward optimality.

Its major limitation is that it generally requires a known environment model and can become computationally expensive for large state spaces.

2. Deep Q-Networks for Intelligent Decision-Making

1. Introduction

A **Deep Q-Network (DQN)** combines **Q-learning** with a deep neural network.

Traditional Q-learning stores action values in a Q-table, which becomes impractical when the state space is very large.

DQN solves this problem by using a neural network to approximate:

$$Q(s, a; \theta)$$

where θ represents the network parameters.

2. Problem Identification

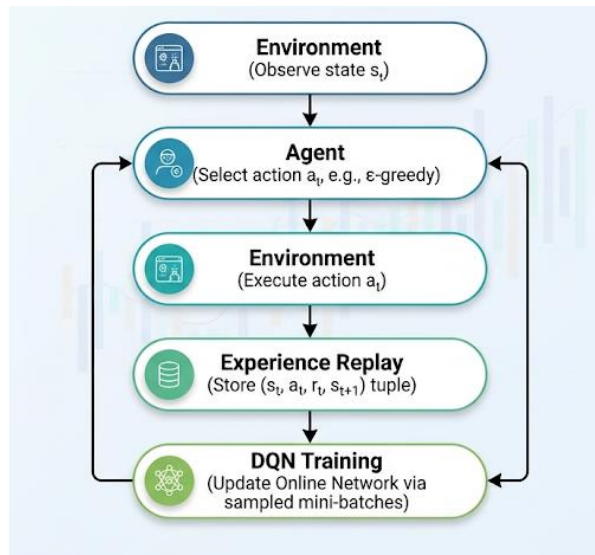
Traditional Q-learning faces difficulties when:

- The state space is very large.
- States are continuous or high-dimensional.
- A Q-table requires excessive memory.
- Learning becomes computationally expensive.

For example, an autonomous robot may receive camera or sensor observations containing thousands of possible states. Maintaining a Q-table for all states is impractical.

DQN uses a neural network to generalize across states.

3. DQN Architecture



The neural network receives a state and outputs Q-values for possible actions.

4. Q-Learning Foundation

The Q-learning target is:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

The network attempts to minimize the difference between the predicted Q-value and target value.

A commonly used loss is:

$$\mathcal{L}(\theta) = (y - Q(s, a; \theta))^2$$

5. Experience Replay

DQN stores experiences in a replay buffer:

$$(s, a, r, s')$$

Instead of training only on the latest experience, the algorithm randomly samples mini-batches from the buffer.

Advantages:

- Reduces correlation between consecutive experiences.
- Improves data efficiency.
- Reuses previous experiences.
- Makes training more stable.

6. Target Network

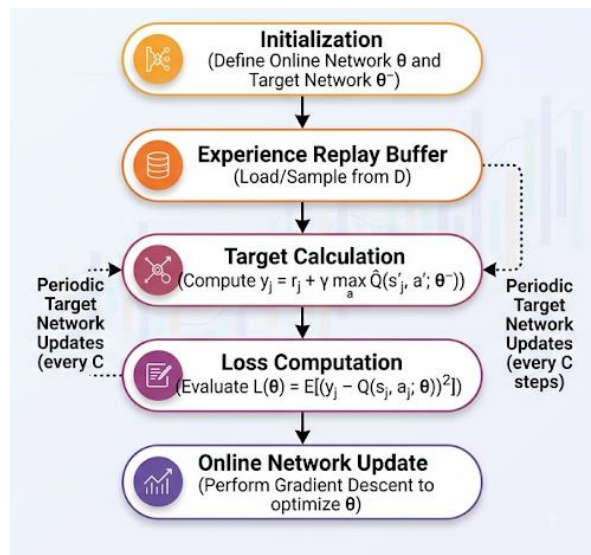
DQN uses two networks:

- **Online network** – updated frequently.
- **Target network** – updated periodically.

The target network provides more stable target values.

This reduces the problem of continuously changing targets during training.

7. DQN Learning Process



8. Exploration and Exploitation

DQN commonly uses **ϵ -greedy exploration**.

With probability ϵ , the agent selects a random action.

With probability $1-\epsilon$, it selects the action having the highest predicted Q-value.

Initially:

High exploration → Learn about environment

Later:

Lower exploration → Exploit learned knowledge

9. Industrial Applications

DQN can be applied to:

- Autonomous robots
- Industrial automation
- Resource allocation
- Traffic management
- Autonomous navigation
- Game-playing systems
- Smart manufacturing

Example: A warehouse robot can use DQN to learn which movement action should be taken to reach storage locations while avoiding obstacles.

10. Tools and Experimental Evaluation

Recommended tools:

- Python
- PyTorch or TensorFlow
- Gymnasium
- NumPy
- Matplotlib

Possible benchmark:

CartPole-v1

Evaluation parameters:

- Episode reward
- Average reward
- Number of episodes
- Training loss
- Convergence behavior

11. Advantages and Limitations

Advantages:

- Handles large state spaces.
- Uses neural networks for generalization.
- Experience replay improves stability.
- Target networks reduce training instability.

Limitations:

- Requires significant computation.
- Sensitive to hyperparameters.
- Standard DQN is mainly designed for discrete actions.
- Training can be unstable without suitable techniques.

12. Conclusion

DQN extends traditional Q-learning to high-dimensional problems by replacing the Q-table with a deep neural network. **Experience Replay and Target Networks** are two important mechanisms that make learning more stable and efficient.

3. Policy-Based Reinforcement Learning for Continuous Control Problems

1. Introduction

Policy-Based Reinforcement Learning directly learns a policy that maps states to actions.

Instead of learning a Q-table first, the algorithm optimizes policy parameters:

$$\pi_{\theta}(a | s)$$

where θ represents the policy parameters.

This approach is particularly useful for **continuous control problems** such as robotics and autonomous driving.

2. Problem Identification

Many real-world systems require continuous actions.

Examples:

- Robot joint angles
- Steering angle
- Vehicle acceleration
- Robotic arm movement
- Motor control

For example, an autonomous vehicle may need a steering angle of **12.7°**, rather than selecting only "left" or "right".

Policy-based RL is suitable because it can directly generate continuous actions.

3. Policy Representation

The policy can be:

Deterministic:

$$a = \pi_{\theta}(s)$$

Stochastic:

$$a \sim \pi_{\theta}(a | s) = \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}^2(s))$$

A stochastic policy provides a probability distribution over possible actions.

4. Policy Gradient

The objective is to maximize expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$$

The policy gradient is:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t \right]$$

where $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$ is the total return from step t .

The parameters are updated using:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

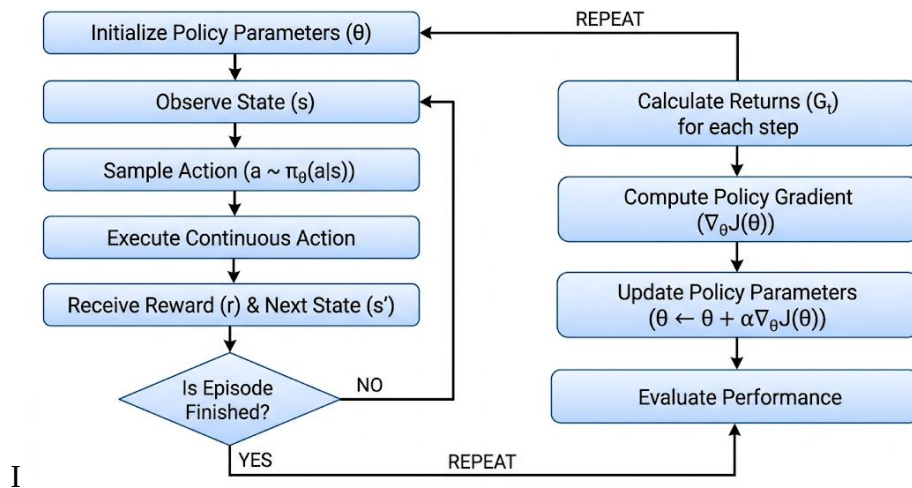
5. REINFORCE Algorithm

REINFORCE is a Monte Carlo policy-gradient method.

Steps:

1. Initialize policy parameters.
2. Generate an episode using the current policy.
3. Collect states, actions, and rewards.
4. Calculate return (G_t).
5. Calculate the policy gradient.
6. Update policy parameters.
7. Repeat until performance converges.

6. REINFORCE Flow



7. Stochastic Policy Search

Stochastic policy search explores different actions according to their probabilities.

This naturally provides exploration because the agent does not always select the same action.

It is useful when:

- The environment is uncertain.
- Multiple actions can be effective.
- Exploration is important.
- Actions are continuous.

8. Applications

Policy-Based RL is useful in:

- Robotic arm control
- Autonomous driving
- Drone navigation
- Industrial automation
- Humanoid robotics
- Continuous motor control

Example: In autonomous driving, the policy can directly determine steering and acceleration values based on road conditions and vehicle state.

9. Advantages

- Works well with continuous action spaces.
- Directly optimizes the policy.
- Naturally supports stochastic behavior.
- Can represent complex decision-making strategies.

10. Limitations

- Policy-gradient estimates can have high variance.

- Training can require many samples.
- Hyperparameter selection is important.
- Basic policy-gradient methods may converge slowly.

11. Tools and Evaluation

Tools:

- Python
- PyTorch/TensorFlow
- Gymnasium
- NumPy
- Matplotlib

Evaluation metrics:

- Average episode reward
- Policy loss
- Convergence rate
- Action quality
- Training stability

12. Conclusion

Policy-Based RL directly optimizes the policy instead of relying only on value estimation. REINFORCE and policy-gradient methods are especially useful for continuous control, making them suitable for robotics, autonomous vehicles, and industrial automation.

4.Model-Based Reinforcement Learning for Autonomous Decision-Making

1. Introduction

Model-Based Reinforcement Learning learns or uses a model of the environment to predict the consequences of actions.

The agent uses this model for **prediction, planning, and decision-making** before executing actions in the real environment.

The basic framework is:

Environment → Model → Planning → Policy → Action

2. Problem Identification

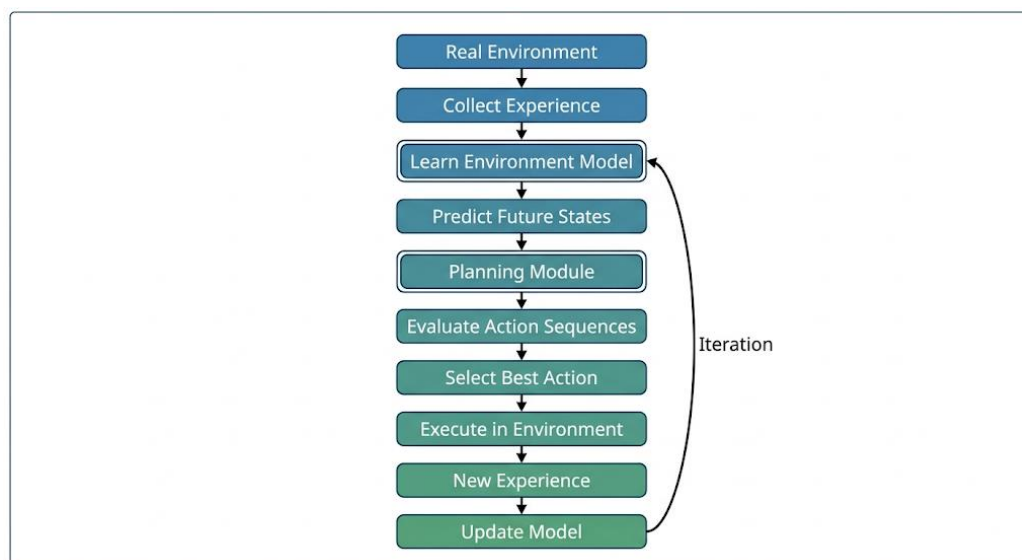
Real-world autonomous systems often have limited opportunities for trial-and-error because mistakes can be expensive or dangerous.

Examples:

- Autonomous robots
- Self-driving vehicles
- Smart manufacturing
- Industrial robots

A model-based agent can simulate possible actions before executing them.

3. Model-Based RL Framework



4. Environment Modelling

The learned model estimates:

$P(s' | s, a)$ and the reward $R(s, a)$ or $r(s, a, s')$

The model therefore predicts:

Current state + Action → Next state + Reward

For example:

Robot position + Move Forward → Predicted new position + movement reward

5. Planning Strategies

The model can be used for:

- One-step prediction
- Multi-step rollout
- Tree search
- Sampling-based planning
- Model predictive control

The planner evaluates possible future outcomes and selects the action with the highest expected return.

6. Prediction

A model can estimate future outcomes:

$$\hat{s}_{t+1} = f_{\theta}(s_t, a_t)$$

The agent can use these predictions to determine which actions are likely to produce higher rewards.

7. Advantages

- Better sample efficiency.
- Can learn from simulated experience.
- Reduces dependence on real-world interaction.
- Supports planning before action execution.
- Useful when real-world experimentation is costly.

8. Limitations

- Learning an accurate model can be difficult.
- Model errors can accumulate during long simulations.
- Planning can require significant computation.
- Incorrect predictions can lead to poor decisions.

9. Industrial Applications

Autonomous Robots

A robot can simulate possible movements and select a collision-free path before executing it.

Smart Manufacturing

The system can predict machine states and production outcomes before changing operating parameters.

Autonomous Vehicles

The model can predict the consequences of braking, acceleration, or steering decisions.

10. Tools and Experimental Evaluation

Tools that can be used:

- Python
- NumPy
- Gymnasium
- PyTorch/TensorFlow
- Matplotlib

Evaluation metrics:

- Prediction accuracy
- Cumulative reward
- Planning time
- Sample efficiency
- Task completion rate

11. Model-Based vs Model-Free

Feature	Model-Based RL	Model-Free RL
Environment model	Yes	No
Planning	Yes	Usually limited
Sample efficiency	Generally higher	Generally lower
Computation	Higher	Lower
Model error	Important issue	Not applicable

12. Conclusion

Model-Based RL is valuable for autonomous decision-making because it allows an agent to **predict, plan, and optimize actions before executing them**. It is particularly suitable for applications where real-world interaction is expensive, risky, or time-consuming.

5.Sampling-Based Planning and Model-Based Policy Optimization

1. Introduction

Sampling-Based Planning is a Model-Based RL technique in which the agent generates multiple possible future trajectories using a learned environment model.

Instead of evaluating every possible action sequence, the agent **samples a manageable number of trajectories** and selects promising actions.

Model-Based Policy Optimization extends this idea by using model-generated experiences to improve the policy.

2. Problem Identification

Traditional RL may require a large number of real interactions.

This can be expensive in:

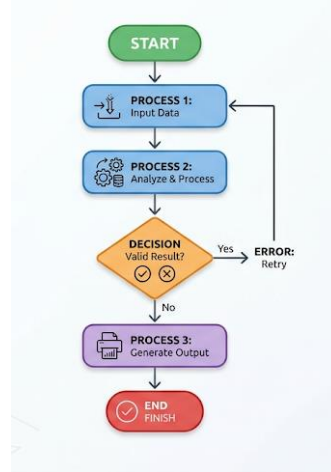
- Robotics
- Manufacturing
- Autonomous vehicles

- Industrial control

The objective is therefore to generate useful additional experiences using a learned model.

3. Sampling-Based Planning

The process is:



4. Sampled Trajectories

A trajectory can be represented as:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$$

The return is:

$$G_t = \sum_{k=0}^T \gamma^k r_{t+k}$$

The planner compares the returns of sampled trajectories and chooses the most promising one.

5. Model-Based Data Generation

The learned model generates synthetic experiences:

$$(s, a, r, s')$$

These experiences can be added to a replay buffer and used to train the policy.

Real Data → Learned Model → Synthetic Data → Policy Training

This improves **sample efficiency** because the agent can learn from both real and simulated experiences.

6. Value-Equivalence Prediction

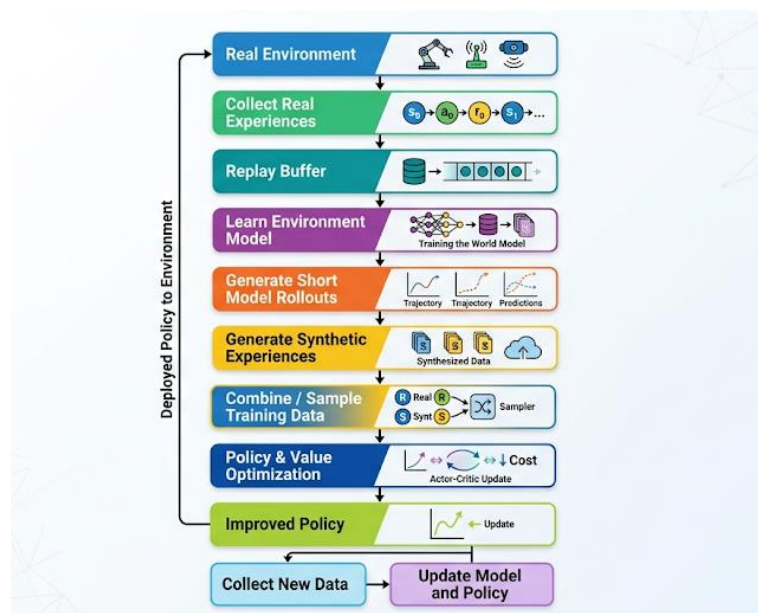
Value-equivalence focuses on whether a learned model provides predictions that are sufficiently useful for **value estimation and decision-making**, even if it does not perfectly reproduce every detail of the real environment.

The objective is to obtain reliable estimates of $V^\pi(s)$ OR $Q^\pi(s, a)$ for making good decisions.

Thus, the model does not necessarily need to perfectly reproduce the entire environment if it provides sufficiently accurate predictions for policy optimization.

7. Model-Based Policy Optimization

The overall process can be represented as:



8. Why Short Rollouts Are Useful

Long model-generated trajectories can accumulate prediction errors.

Therefore, using **short model rollouts** can reduce the effect of model inaccuracies.

This provides a balance between:

More synthetic data ↔ Model prediction reliability

9. Advantages

- Improves sample efficiency.
- Reduces expensive environment interactions.
- Enables planning using simulated trajectories.
- Can accelerate policy learning.
- Useful for real-world control applications.

10. Limitations

- Synthetic data may contain model errors.
- Poor models can produce misleading experiences.
- Long rollouts may accumulate errors.
- Model learning and planning add computational cost.

11. Industrial Case Study – Autonomous Robot

Consider a warehouse robot that needs to move goods between storage locations.

The learned model predicts how the robot moves under different actions. The planner samples possible movement sequences and predicts their future rewards.

The robot selects the trajectory with:

- Shorter travel time
- Lower collision risk
- Lower energy consumption

The resulting real experience is used to improve the model and policy.

Thus, the system gradually becomes more efficient without requiring excessive real-world experimentation.

12. Tools and Experimental Evaluation

Possible tools:

- Python
- NumPy
- PyTorch/TensorFlow
- Gymnasium
- Matplotlib

Evaluation parameters:

Parameter	Purpose
Cumulative Reward	Measures policy quality
Sample Efficiency	Measures learning per environment interaction
Planning Time	Measures computational cost
Model Prediction Error	Measures model quality
Task Success Rate	Measures practical performance

13. Conclusion

Sampling-Based Planning and Model-Based Policy Optimization combine **environment modelling, trajectory sampling, synthetic data generation, value prediction, and policy optimization**.

They are especially useful for real-world applications where collecting large amounts of real interaction data is expensive or risky.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism ; uneven team participation	Lacks professionalism ; minimal contribution or ethical awareness